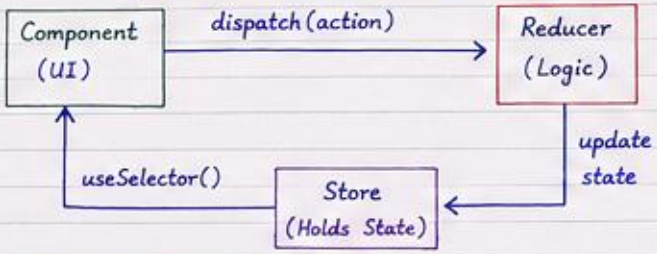
☆ REDUX TOOLKIT (RTK) - TOPPER NOTES ☆

1. What is Redux ?

Redux is a state management library for JavaScript apps.
It stores the entire application state in a single central store.

Redux Flow (Mental Model)



2. Core Building Block - SLICE

A slice is a collection of :

- initial state
- reducers
- generated action creators
- other reducer logic

★ createSlice() from @reduxjs/toolkit is used to create a slice.

3. Key Terms

- Store → central place that holds the state
- Slice → state + reducers + actions
- Reducer → function that decides how state changes based on action
- Action → object that describes what happened
- Dispatch → sends an action to the store
- Selector → reads data from the store
- Payload → data passed with an action
- Immer → RTK uses Immer internally to write "mutating" code safely.

4. createSlice() – Example (counterSlice.js)

```
import { createSlice } from "@reduxjs/toolkit"

const counterSlice = createSlice({
  name : 'counter',
  initialState : {
    value : 0,
    name : "Kapil"
  },
  reducers : {
    increment : (state) => {
      state.value += 1
    },
    decrement : (state) => {
      if(state.value > 0) state.value -= 1
    },
    reset : (state) => {
      state.value = 0
    },
    changeName : (state, action) => {
      state.name = action.payload
    }
  }
})

export const { increment, decrement, reset, changeName }
                = counterSlice.actions
export default counterSlice.reducer
```

We can write mutating code like state.value++ because Immer handles immutability internally.

5. Configure Store (store.js)

```
import { configureStore } from '@reduxjs/toolkit'
import counterReducer from './features/counter/counterSlice'

const store = configureStore({
  reducer : {
    counter : counterReducer      // key = "counter"
  }
})
export default store
```

State structure will be:
state.counter.value
state.counter.name

6. Provide Store to React App – Provider

```
import { Provider } from 'react-redux'
import store from './store'
import App from '../App'

<Provider store={store}>
  <App />
</Provider>
```

Provider makes the store available to all components in the app.

7. Using Redux in Components

A. useSelector() – Read State

```
const value = useSelector(
  (state) => state.counter.value
)
```

useSelector always takes the latest state from the store.

B. useDispatch() – Send Action

```
const dispatch = useDispatch()
```

- dispatch(increment())
- dispatch(decrement())
- dispatch(reset())
- dispatch(changeName("Rahul"))

Action Creators (from slice)

8. Action & Payload

When we pass some data while dispatching an action, it is available in reducer as **action.payload**

dispatch(changeName('Rahul')) → Reducer

state.name = action.payload ('Rahul')

☆ Payload helps us send data with an action.

9. Complete Flow

Component dispatches an action
↓
Action (with type + payload) goes to the store
↓
Reducer (from slice) receives action
↓
Reducer updates the state
↓
Updated state is saved in the store
↓
Components using useSelector() re-render
↓
UI is updated

☆ Golden Rules

① READ → useSelector()
② WRITE → useDispatch()
③ ACTION → describes what happened
④ PAYLOAD → data with action
⑤ REDUCER → updates state
⑥ SLICE → state + reducers + actions
⑦ IMMER → safe mutating code in reducers

★ Understand the flow, not just the code.
That's the key to mastering Redux Toolkit! ☺ ★

11. Quick Revision CheatSheet

- createSlice() → create slice (state + reducers + actions)
- configureStore() → create store and add reducers
- Provider → makes store available in app
- useSelector() → read state
- useDispatch() → send action
- action.payload → carries data
- Immer → write mutating code safely

Example Usage in Component

```
const value = useSelector(
      (state) => state.counter.value
)
const dispatch = useDispatch()

return (
  <div>
     <p>Count : {value}</p>
     <button onClick={() => dispatch(increment())}></button>
     <button onClick={() => dispatch(decrement())}></button>
     <button onClick={() => dispatch(reset())}>Reset</button>
  </div>
)
```

Remember

Redux Toolkit reduces boilerplate and makes Redux simple, powerful and enjoyable!